StormVPN version 0.1.1

client-server application for building an Ethernet tunnel over HTTPS

user’s manual

Table of contents:

Introduction

StormVPN is proprietary software designed to build Ethernet tunnels over an IPv4 network (the Internet) between Linux operating systems. This software is built on a client-server model, where the server acts as either a standalone HTTPS server or provides HTTP for finding an HTTPS proxy. The latter scheme can be used when you want to place a tunnel together with a website or websites running behind a reverse proxy. Although StormVPN's ideology implies the transmission of Ethernet traffic, the package also includes Windows (IP only) and Android (IP only) clients, in which part of the operating system functionality (L2-L3) is built directly into the application itself, such as ARP resolving. StormVPN has all the modern methods of traffic management and control, from bridging and VLAN filtering, as well as their re-tagging, to DSCP traffic marking and bandwidth control. It should also be noted that in addition to the classic Ethernet mode, StormVPN has IP modes, including Dynamic, which allow you to quickly set up a BGP/OSPF-like session without the need for third-party software.

The server program and client program do not require any external software, including ifconfig, ip, brctl, and vlan. However, the following software has been added to the StormVPN-Server Docker image for x86_64 (amd64): tzdata, bash, iproute2, bird, grep, gawk. This provides the minimum necessary additional functionality, such as isolating the StormVPN server in IP mode within a separate namespace with BGP protocol support. This functionality is not part of the StormVPN package but can be implemented using an additional wrapper script.

1. Server description

The server is a binary file for Linux/ELF (stormvpn-server or server_static) that has the following command line parameters:

```
-notimeinlog
      Don't add timestamps to log
-reload
      Send SIGHUP to running server process to reload configuration
```

As you can see, the server supports soft reconfiguration by sending SIGHUP, which will reread all configuration files and, if possible, apply changes without restarting.

1.1. Main configuration file

It should be noted right away that neither the server nor the client keep log files, all logging is done on the console.

When started, the server reads the configuration file /etc/storm-vpn/server.conf, which consists of the following configurable parameters:

listen_ip - the address where the server will listen, 0.0.0.0 - default;
standalone - true or false defines the server operation mode, in case of standalone=true the server will work on https_port and perform encryption on its own, server_crt and server_key will also be used. This mode implies that the server works independently - without web services on one IP. In the case of standalone=false, the server works on the http_port and essentially means working behind an HTTP(s) proxy, such as Nginx;
url - suffix of HTTP URL to connect to VPN, all other paths will give 404 error, full path to connect will look like https://storm-vpn.your-domain.com/<url> (storm-vpn-endpoint - default);
client_timeout - session timeout in seconds, when this timeout expires and there is no data and Keepalive(s) the session will be terminated;
http_port - port for HTTP, 80 - default;
https_port - port for HTTPS, 443 - default;
server_crt - PEM certificate for HTTPS, /etc/storm-vpn/server.crt - default;
server_key - PEM/key for HTTPS, /etc/storm-vpn/server.key - default;
serial - serial number of the program, demo - default.

By default, the server will be launched as follows: 0.0.0.0:443/storm-vpn-endpoint, i.e., preference is given to the standalone=true mode. In this case, you must have server_key and server_crt files.

Comments beginning with the “#” symbol may be used in the configuration file.

Serial must be either your serial number (64 bytes) issued to you or demo for Demo mode.

1.2. Peers configuration file

The file /etc/storm-vpn/peers.conf describes your clients that will connect to your server in the following format:

```
[peer_name]
other_options
```

Minimum required configuration for each peer:

```
[peer_name]
secret_key=password
interface_name=iface_name
```

This essentially describes the login, password, and name of the interface that will be created on the Linux server running StormVPN-Server.

It should be noted right away that in this case, an Ethernet interface (tap) will be created, which will have a random MAC address, will not be connected to the bridge, and will not have IP settings.

Attention! For the Linux server and client to work, you need the tun kernel module. Make sure it is loaded before you start using the software.

Other parameters for /etc/storm-vpn/peers.conf:

mac_address - MAC address of the TAP interface, default – auto;
link_quality - speed limit for continuous measurement (similar to iperf), specified in the form <INT>(K|M), i.e. link_quality=512K or link_quality=2M, where K is kilobits and M is megabits. In this case, the measurement will be performed at a speed not exceeding the specified value. Alternatively, you can specify auto, in which case the measurement will be performed at the maximum available channel speed. Starting with version 0.1.0, in auto mode, measurements are only taken when there is no useful (DATA) traffic, while when a limit is set, measurements are taken continuously;

ban_quality - (since version 0.1.0) If, when measuring link_quality (not in auto mode), the speed drops below ban_quality in the form of <INT>(K|M), the peer is blocked from transmitting useful data, the connection remains, but the data in it stops flowing until link_quality returns to its normal value;

link_type - connection type, can take the following values: none, bridge, or ip. For bridge and ip, additional parameters must be specified;

bridge_vlan - a parameter required for VLAN filtering in Bridge connection mode;

bandwidth - client speed limit in K - kilobits or M - megabits, set similarly to link_quality, i.e. bandwidth=3M will limit the client to 3 MB/s for downloads and 3 MB/s for uploads.

1.2.1. Bridge mode

Bridge mode assumes that you already have a Bridge created, into which you need to place the interface on the server side. This mode also assumes the configuration of a remote (client) Bridge for placing the interface on the client side into it. This mode is useful when you need to pass Ethernet traffic between the server and the client and switch it with the existing infrastructure - other Ethernet interfaces or virtual machine interfaces.

Example:

```
link_type=bridge:bridges:bridgec
```

In this case, **bridges** is the Bridge interface on the server side, into which the interface will be placed, and **bridgec** is the interface that will be transferred to the client so that it can place its own interface there.

Since the Bridge mode supports 802.1q by default, i.e. it supports passing all VLANs, you can also use the bridge_vlan function.

Example:

```
bridge_vlan=15:12
```

In this case, a filter will run on the server side that will discard all tags except 15, and a filter will run on the client side that will discard all tags except 12. In the VPN channel, traffic will move without tags, and the necessary tags 15 and 12 will be set at the edges of the tunnel. This example allows you to re-tag traffic. When the same tag is specified on both sides, simple VLAN filtering will be performed.

1.2.2. IP Mode

IP mode is not an additional mode for the VPN protocol, but simply means that IPv4 will be used on top of pure Ethernet. In IP mode, the server can configure a static

IPv4 configuration for the server and client interfaces, as well as configure a kind of dynamic routing between the client and server without the involvement of additional services such as BGP or OSPF.

The general view of the IP mode is as follows:

```
link_type=ip:NETMASK:SERVER_IP:CLIENT_IP:MODE:METRIC
```

NETMASK - is the subnet mask for the junction between peers, for example 30;
SERVER_IP - is the server-side IP address that will be assigned to the VPN interface;
CLIENT_IP - is the client-side IP address that will be assigned to the VPN interface;
MODE - is the mode of operation: static, dynamic, or full.
(**static** - implies IP assignment to interfaces, **dynamic** implies IP assignment to interfaces and dynamic route exchange during the session, but except for Default, Link-Local and Connected, and **full** implies additionally Connected route exchange. The last mode should be used carefully and most likely you will be satisfied with dynamic for dynamic routing.)
METRIC - is a metric for adding routes on the server and client.

Example:

```
link_type=ip:30:192.168.0.1:192.168.0.2:dynamic:50
```

In this case, the server will be assigned the address 192.168.0.1/30, the client will be assigned 192.168.0.2/30, and dynamic route exchange mode will be enabled, with routes being added with a metric of 50 on both the server and the client.

1.3. Monitoring the server from the console

The StormVPN package also contains a console utility called stats (in the Docker image – stormvpn-stats), which is designed to monitor server via a special socket /var/lib/storm-vpn/storm-vpn.sock.

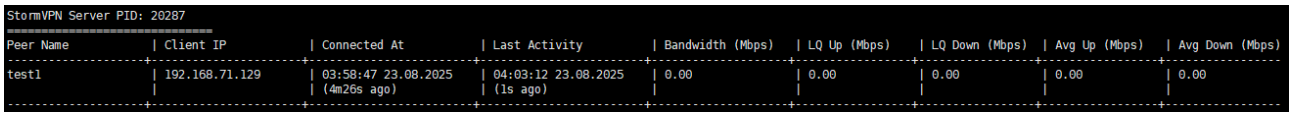

Fig. 1. Stats utility output

Starting with version 0.1.0, the LQ (link_quality) field also reflects the ban status according to ban_quality - BANNED.

The stats utility also supports the kick argument, which allows you to send a command to the server to disconnect a peer. This argument must be executed in the format stats kick <peer_name>.

1.4. Features of Docker image

The basic form of launching a server image is as follows:

```
# docker run --cap-add=NET_ADMIN --device /dev/net/tun:/dev/net/tun --net=host
--name stormvpn-server -d stormotron/stormvpn-server:latest
```

The image contains a special binary file called docker-init, which is designed to configure the server by transferring parameters from the environment directly to the server itself, and also has Web console functionality.
Environment variables:

- **LISTEN_IP** - Server bind IP, default 0.0.0.0
- **HTTP_PORT** - Server HTTP port, default 80
- **HTTPS_PORT** - Server HTTPS port, default 443
- **URL** - Server URL, default storm-vpn-endpoint
- **STANDALONE** - Server MODE, default true
- **CLIENT_TIMEOUT** - Server to client timeout, default 30
- **SERIAL** - Serial or demo, default demo
- **SERVER_CRT** - Server certificate, default selfsigned
- **SERVER_KEY** - String, Server certificate key, default selfsigned
- **CLIENT<NUM>** - clients description, no default - MUST be defined, if WEB interface is disabled
- **WEB_PORT** - Web interface port, enables WEB interface, no default
- **WEB_IP** - Web interface IP, default: 0.0.0.0
- **WEB_PASSWORD** - Web interface password, default admin

Client examples:

```
CLIENT1=peer=peer01,secret=peer01secret
CLIENT2=peer=peer02,secret=peer02secret,interface=peer02,mac=00:11:22:33:44
:55
CLIENT3=peer=bwpeer,secret=bwsecret,bandwidth=10M
CLIENT4=peer=localpeer,secret=localpeer,link_quality=5M,ban_quality=2M
CLIENT5=peer=brpeer,secret=brpeersecret,link_quality=auto,link_type=bridge:my
bridge:clbridge
CLIENT6=peer=brpeer2,secret=brpeersecret2,link_type=bridge:mybridge:clbridge,
bridge_vlan=10:16
CLIENT7=peer=ippeer,secret=ippeersecret,link_type=ip:30:192.168.0.1:192.168.0.
2:dynamic:50
```

Example:

```
# docker run --cap-add=NET_ADMIN --device /dev/net/tun:/dev/net/tun --net=host
-e CLIENT1="peer=test,secret=test" --name stormvpn-server -d
stormotron/stormvpn-server:latest
```

This example will create a server with one configured peer test/test with the interface vpn_1.

If you have not specified a certificate and key and they do not currently exist, docker-init will automatically create a self-signed certificate /etc/storm-vpn/server.crt and key /etc/storm-vpn/server.key.

1.4.1. Using the WEB console

As already mentioned, docker-init supports WEB console for the server. In this case, it will be relevant to use real configuration files as follows:

```
# docker run --cap-add=NET_ADMIN --device /dev/net/tun:/dev/net/tun --net=host -e WEB_PORT=888 -e WEB_PASSWORD=mypass -v /etc/storm-vpn/server.conf:/etc/storm-vpn/server.conf -v /etc/storm-vpn/peers.conf:/etc/storm-vpn/peers.conf --name stormvpn-server -d stormotron/stormvpn-server:latest
```

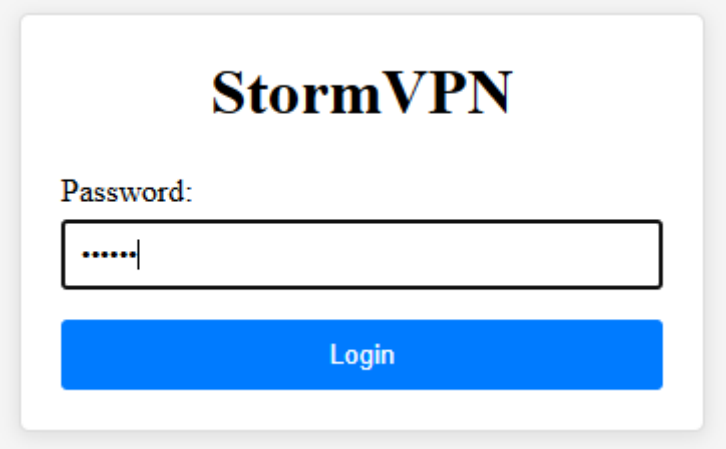

Fig. 2. Web console login window

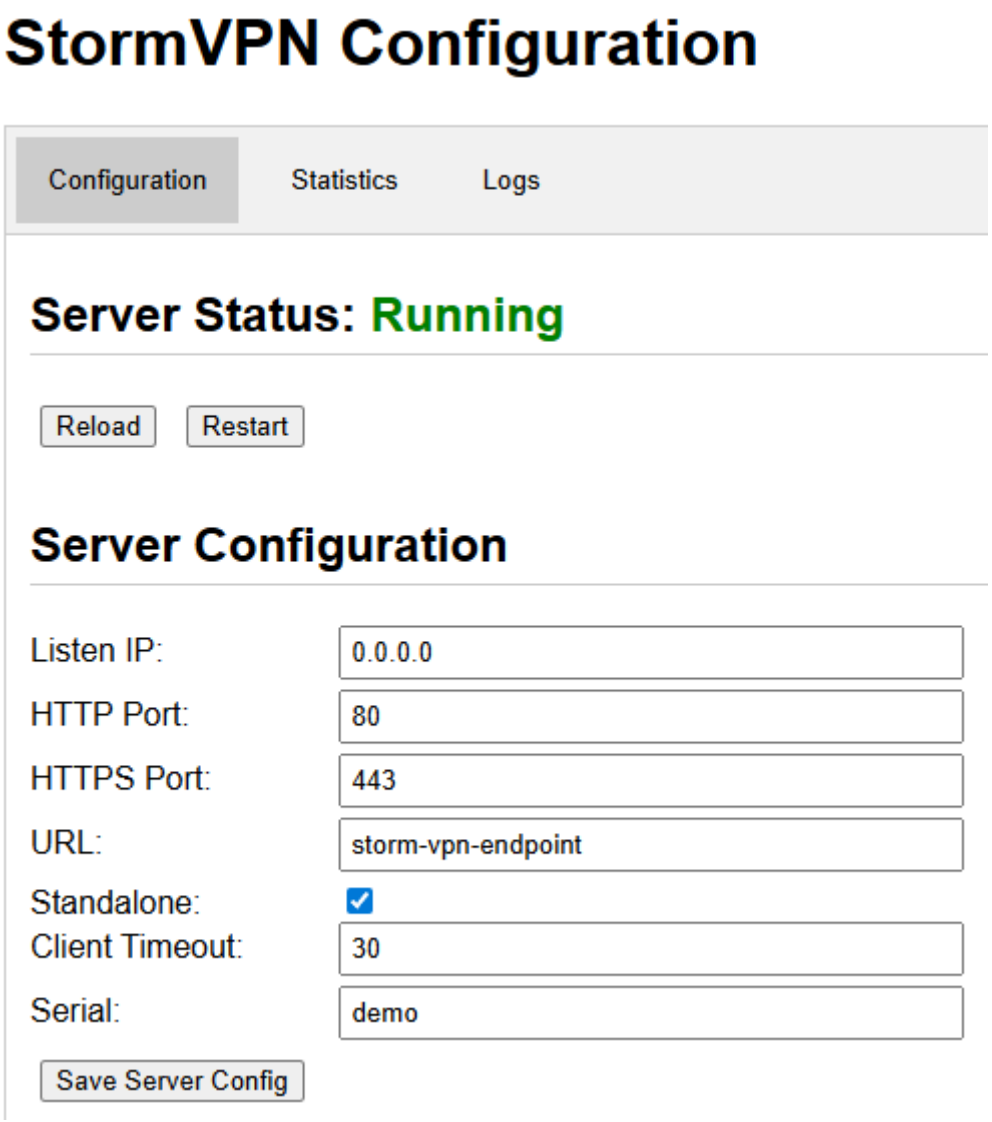

Fig. 3. Configuration window

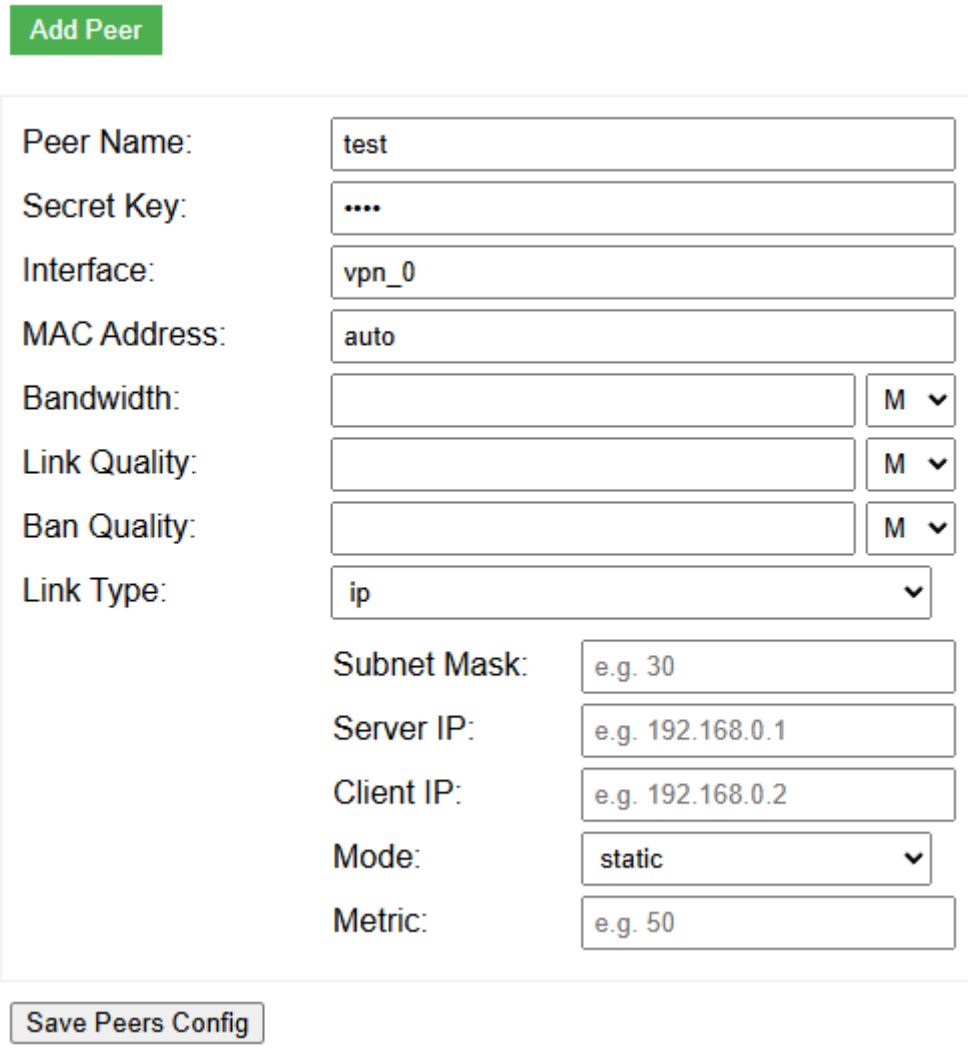

Fig. 4. Peer configuration

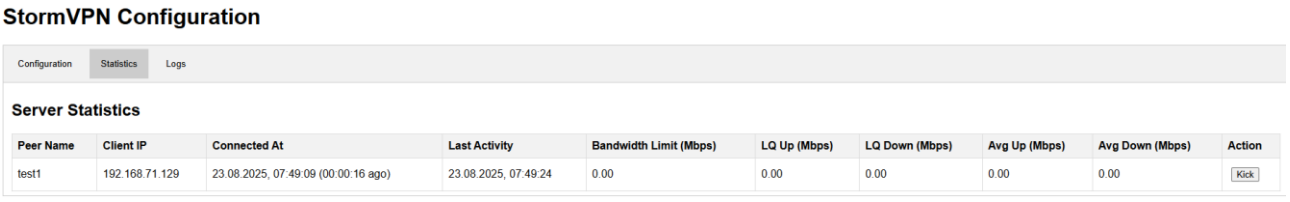

Fig. 5. Statistics

Fig. 6. Logs

1.4.2. Using Docker compose

Example:

```
name: stormvpn-server
services:
  stormvpn-server:
    image: stormotron/stormvpn-server:latest
    cap_add:
      - NET_ADMIN
    devices:
      - /dev/net/tun:/dev/net/tun
    network_mode: host
    environment:
      - STANDALONE=false
      - HTTP_PORT=888
      - URL=custom-storm-vpn-endpoint
      - CLIENT1=peer=tunnel01,secret=SUPERSECRET,interface=tunnel01,link_type=bridge:bridges:bridgec,bridge_vlan=35:41
      - SERIAL=MYSERIAL
    restart: unless-stopped
    container_name: stormvpn-server
```

Example:

```
name: stormvpn-server
services:
  stormvpn-server:
    image: stormotron/stormvpn-server:latest
    cap_add:
      - NET_ADMIN
    devices:
      - /dev/net/tun:/dev/net/tun
    network_mode: host
    environment:
      - WEB_PORT=8080
      - WEB_IP=192.168.84.5
      - WEB_PASSWORD=MY_PASSWORD
    volumes:
      - /opt/stormvpn/configs/server.conf:/etc/storm-vpn/server.conf
      - /opt/stormvpn/configs/peers.conf:/etc/storm-vpn/peers.conf
    restart: unless-stopped
    container_name: stormvpn-server
```

1.4.3. Docker image tricks

As already mentioned, the StormVPN-Server Docker image contains additional software: tzdata, bash, iproute2, bird, grep, gawk. This software can be used by you for special cases. I suggest considering the possibility of building an isolated IP-Only server with BGP.

Wrapper example:

```
#!/bin/bash

set -euo pipefail

# === Configuration ===
NETNS="svpn-server"
BRIDGE="bridge-srv"
```

```
VETH_MAIN="veth-svpn-srv-m"
VETH_NS="veth-svpn-gw"
VLAN_ID="777"
VLAN_IF="veth-svpn-gw-"$VLAN_ID
GW_IP="192.168.91.1"
IP_ADDR="192.168.91.2/29"
LOCAL_BGP_AS=151
REMOTE_BGP_AS=150
# === Configuration ===

write_bird_config() {
local RID

RID=`echo $IP_ADDR | awk -F'/' '{print $1}'`

echo "router id $RID;

protocol device { scan time 5; }

protocol direct {
   ipv4;
}

protocol kernel {
   ipv4 { import none; export all; };
}

filter bgp_import {
   reject;
}

filter bgp_export {
   if net ~ [ 0.0.0.0/0{30,30} ] then accept;
   reject;
}

protocol bgp remote_bgp {
   local as $LOCAL_BGP_AS;
   neighbor $GW_IP as $REMOTE_BGP_AS;

   ipv4 {
      import filter bgp_import;
      export filter bgp_export;
   };
```

```
}" > /etc/bird.conf
}

netns_exists() {
    ip netns list | grep -qw "$NETNS"
}

link_exists() {
    ip link show "$1" &>/dev/null
}

link_exists_ns() {
    ip netns exec "$NETNS" ip link show "$1" &>/dev/null
}

do_start() {
    echo "[+] Starting SVPN-Server..."

    if ! netns_exists; then
        ip netns add "$NETNS"
        echo " +- namespace $NETNS created"
    else
        echo " +- WARNING: namespace $NETNS already exists"
    fi

    if ! link_exists "$VETH_MAIN" && ! link_exists_ns "$VETH_NS"; then
        ip link add "$VETH_MAIN" type veth peer name "$VETH_NS"
        echo " +- veth created"
    else
        echo " +- WARNING: veth already exists"
    fi

    if ! link_exists_ns "$VETH_NS"; then
        ip link set "$VETH_NS" netns "$NETNS"
    fi

    if ! brctl show "$BRIDGE" | grep -qw "$VETH_MAIN"; then
        ip link set "$VETH_MAIN" up
        brctl addif "$BRIDGE" "$VETH_MAIN"
        echo " +- $VETH_MAIN added to $BRIDGE"
    fi

    ip netns exec "$NETNS" ip link set "$VETH_NS" up
```

```
    if ! link_exists_ns "$VLAN_IF"; then
        ip netns exec "$NETNS" ip link add link "$VETH_NS" name "$VLAN_IF"
type vlan id "$VLAN_ID"
        echo " +- created VLAN $VLAN_IF"
    fi

    ip netns exec "$NETNS" ip link set "$VLAN_IF" up

    if ! ip netns exec "$NETNS" ip addr show "$VLAN_IF" | grep -qw
"$IP_ADDR"; then
        ip netns exec "$NETNS" ip addr add "$IP_ADDR" dev "$VLAN_IF"
    fi

    ip netns exec "$NETNS" ip route del default 2>/dev/null || true
    ip netns exec "$NETNS" ip route add default via "$GW_IP"

    write_bird_config
    ip netns exec "$NETNS" /usr/sbin/bird -c /etc/bird.conf -s /run/bird.ctl
    ip netns exec "$NETNS" /bin/docker-init
}

do_stop() {
    echo "[+] Stopping SVPN-Server..."

    if netns_exists; then
        if link_exists_ns "$VLAN_IF"; then
            ip netns exec "$NETNS" ip link delete "$VLAN_IF" || true
        fi

        if link_exists "$VETH_MAIN"; then
            ip link delete "$VETH_MAIN" || true
        fi

        ip netns del "$NETNS" || true
        echo " +- namespace $NETNS deleted"
    fi
}

set +u
case "$1" in
start)
    set -u
    do_start
    ;;
```

```
stop)
    set -u
    do_stop
    ;;
*)
    echo "Usage: $0 {start|stop}"
    exit 1
    ;;
esac
```

This wrapper creates a separate namespace inside the Docker container - svpn-server. The interface created inside the namespace will be based on a veth pair from the main system (bridge-srv) with VLAN id=777, and this new interface will be assigned IP 192.168.91.2/29. The default gateway will be 192.168.91.1. Connected clients (IP mode, /30) will be advertised to the BGP neighbor (AS150).

Compose example:

```
name: stormvpn-server
services:
    stormvpn-server:
        image: stormotron/stormvpn-server:latest
        cap_add:
            - NET_ADMIN
            - SYS_ADMIN
        devices:
            - /dev/net/tun:/dev/net/tun
        volumes:
            - /opt/stormvpn-server/bin/wrapper.sh:/bin/wrapper.sh
        network_mode: host
        environment:
            - STANDALONE=false
            - URL=my-vpn-endpoint
            - LISTEN_IP=192.168.91.2
            - HTTP_PORT=10300
            - CLIENT1=peer=test1,secret=test1,interface=sv-test1,link_type=ip:30:192.168.99.17:192.168.99.18:static:50
        restart: unless-stopped
        container_name: stormvpn-server
        command: [ "/bin/wrapper.sh", "start" ]
```

2. Linux client description

The Linux client can only be configured using the command line:

```
  -I      Ignore TLS certificate verification errors
  -d int
        DSCP value for outgoing traffic (0-63)
  -i string
        TUN/TAP interface name (max 15 chars)
  -k int
        Keepalive interval in seconds (1-60) (default 5)
  -m string
```

```
        MAC address for TAP interface ('auto' or specific MAC) (default "auto")
  -notimeinlog
        Don't add timestamps to log
  -p string
        Peer name (must match server config)
  -r int
        Max connection retries (0=infinite, 1-999)
  -s string
        Secret key (must match server config)
  -u string
        Server URL (e.g., https://vpn.example.com/storm-vpn-endpoint)
  -x string
        Proxy server (user:password@ip:port)
```

The only notable features are the DSCP parameter and the possible use of a SOCKS5 proxy to reach the server. DSCP allows you to assign a label to outgoing traffic so that other equipment (routers, switches) can prioritize this traffic.

Client environment variables for Docker Image:

- **URL** - Server URL, default https://vpn.your.domain/storm-vpn-endpoint
- **PEER** - Peer name, default test
- **SECRET** - Secret, default test
- **INTERFACE** - Local interface, default vpn_<RANDOM NUMBER>
- **INSECURE** - Is certificate verification enabled, default false
- **PROXY** - SOCKS5 proxy, format: user:password@ip:port, no default value
- **DSCP** - DSCP mark, no default value

3. Other clients

3.1. Windows client

The Windows client contains an OpenVPN TUN/TAP driver for Windows, through which the client is supposed to work. However, only IP mode is supported, as Bridge mode does not make sense for Windows. The client is implemented as a console application with a separate GUI interface that supports the creation and use of connection profiles.

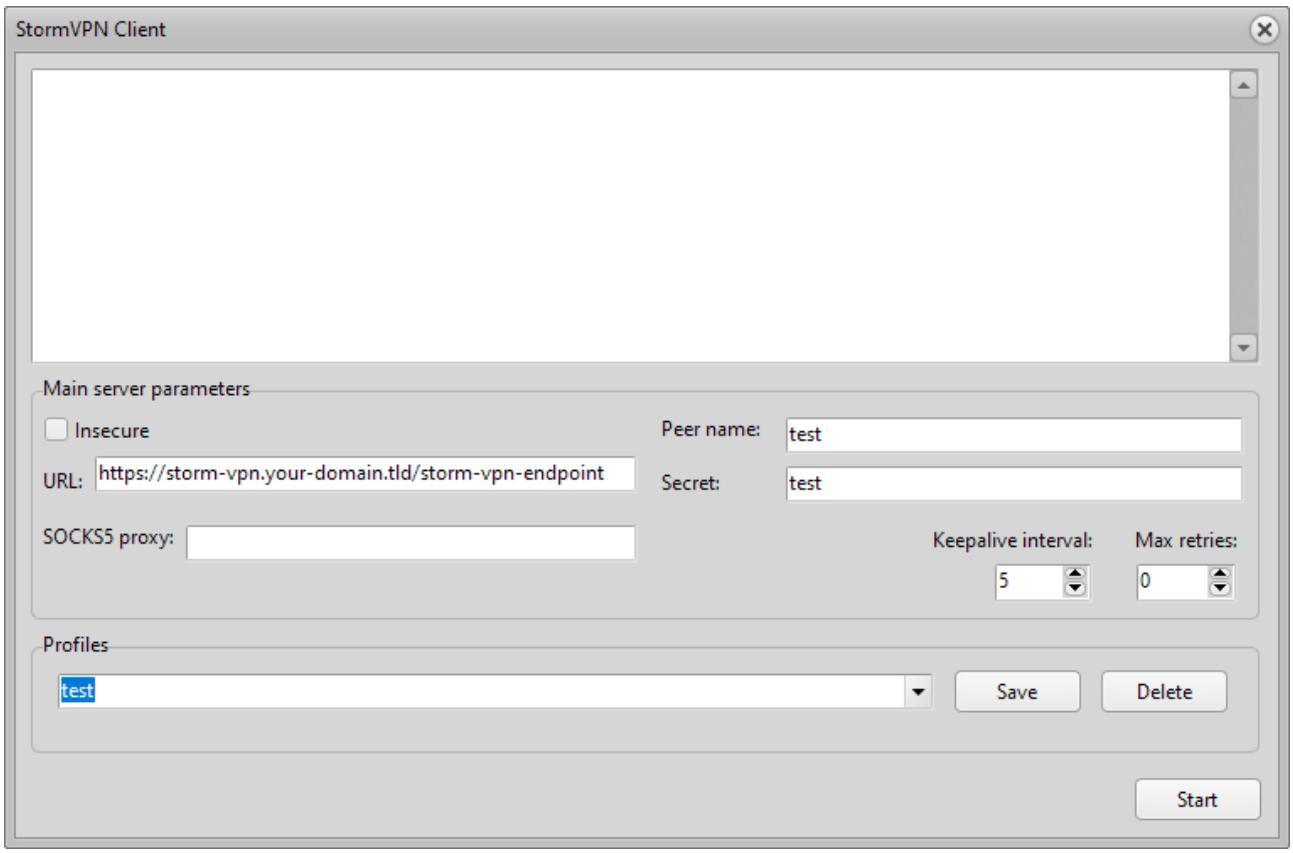

Fig. 7. Windows GUI IP-only client

3.2. Android client

The Android client is similar to the Windows client. However, there are some minor differences due to the fact that Android does not support Ethernet tunneling in principle, so functionality such as ARP resolution (L2.5) is built directly into the client.

Fig.8 Android VPN client

4. Licensing

Without the correct key, the server operates in limited mode and can only accept one connection. Also, starting with version 0.0.9, the unregistered version of the server has a session duration limit of 10 minutes. If you need more than one connection to the server or the session should not be interrupted, you will need a key. To obtain a key, you can contact us by providing the request code received when starting the unregistered server. Starting with version 0.0.7, a one-week trial key has been added to the licensing model, which you can order for free by contacting us. **Please order a free trial key before purchasing a full key to see if the product meets your expectations**.

Starting with version 0.1.1, the server-side protection of the program has been updated, new serial keys are now in effect, and the request code format has changed. The old keys are no longer valid. The signature keys for the Android application have also changed.

Please contact us for details regarding licensing and product pricing.

If you need a free alternative that does not require the aforementioned functionality, you can consider our other project, Narnia - https://github.com/stormotron/narnia .

To contact me, please use Telegram: el_stormi (http://t.me/el_stormi).